

Sistema Inteligente de Monitoreo de Nivel de Agua

Cristian De Jesús Ortega Montes

Javier De Jesús Ortega Montes

Brayan Andrés Bertel Álvarez

Docente: José López Prado

Universidad de Sucre

Facultad de Ingeniería

Ingeniería Electrónica

Octavo semestre

Sincelejo - Sucre

2025

Resumen

Este proyecto consiste en el desarrollo de un sistema de monitoreo en tiempo real del nivel de agua en un tanque, así como el cálculo automático del consumo expresado en litros. Para ello, se utiliza un sensor ultrasónico conectado a un microcontrolador ESP32, el cual permite medir la distancia entre el sensor y la superficie del agua. Con base en esta medición y las dimensiones del tanque (altura y diámetro), el sistema calcula el volumen actual de agua en litros. Es compatible con tanques de geometría cilíndrica o cónica. A partir de este cálculo, también se determina el porcentaje de llenado del tanque. Los datos obtenidos son enviados mediante una conexión Wi-Fi a una API, la cual se encarga de registrar la información en una base de datos MongoDB. Para la visualización de los datos en tiempo real y el historial de consumo, se desarrolló una aplicación móvil utilizando Expo Go (React Native). La aplicación le permite al usuario configurar parámetros esenciales como: dimensiones del tanque (altura, diámetro, tipo de geometría), parámetros de la conexión a la red Wi-Fi y la tarifa de agua por metro cubico establecida por la empresa prestadora del servicio. Además el usuario puede consultar el volumen de agua disponible en litros, porcentaje de llenado, historial de consumo diario, semanal y mensual y el costo del consumo en función de la tarifa configurada.

El sistema resuelve la necesidad de monitorear y gestionar eficientemente el nivel de agua en tanques de almacenamiento, permitiendo conocer en tiempo real el volumen disponible, el porcentaje de llenado y el consumo histórico. Esto ayuda a prevenir desabastecimientos, detectar fugas o consumos anómalos, y controlar los costos asociados al uso del agua, especialmente en contextos donde el recurso hídrico es limitado o su consumo representa un gasto significativo.

Está dirigido a usuarios residenciales, agrícolas, industriales o institucionales que dependen del almacenamiento de agua en tanques para su operación diaria. Esto incluye hogares, fincas, instalaciones rurales, pequeños negocios, plantas de producción, centros educativos o cualquier entidad que requiera supervisar el uso del agua y mantener un control eficiente sobre este recurso.

Se utilizó un microcontrolador ESP32 con conectividad Wi-Fi, un sensor ultrasónico para la medición de distancia, una API para el procesamiento de datos, y una base de datos NoSQL MongoDB para el almacenamiento. La aplicación móvil fue desarrollada con React Native utilizando Expo Go, y la comunicación entre dispositivos se realiza mediante el protocolo HTTP.

Introducción

El acceso, uso eficiente y monitoreo del recurso hídrico se ha convertido en una necesidad crítica en diversos sectores, especialmente ante el creciente impacto del cambio climático y el aumento en la demanda de agua a nivel global. En contextos rurales, residenciales e industriales, el uso de tanques de almacenamiento de agua es una práctica común, sin embargo, la mayoría de estos sistemas carecen de mecanismos de monitoreo en tiempo real que permitan conocer con precisión el volumen de agua disponible, el consumo diario y posibles pérdidas por fugas.

Este proyecto propone el diseño e implementación de un sistema inteligente de monitoreo de nivel de agua en tiempo real, utilizando tecnologías accesibles y de bajo consumo energético. El sistema, basado en un sensor ultrasónico y un microcontrolador ESP32 (Espressif Systems, 2022) con conectividad Wi-Fi, permite medir la distancia desde el sensor al nivel del agua en un tanque, calcular el volumen restante en litros y enviar esta información a una plataforma en línea mediante una API. Los datos son almacenados en una base de datos MongoDB (MongoDB Inc, 2024) y visualizados a través de una aplicación móvil desarrollada en React Native (Meta, 2024). con Expo Go (Expo, 2024).

Justificación

La gestión adecuada del agua es esencial para la sostenibilidad ambiental, económica y social. Según la Organización de las Naciones Unidas para la Alimentación y la Agricultura (FAO, 2023), una gestión eficiente del agua es clave para garantizar la seguridad hídrica global. Implementar soluciones de monitoreo digital permite no solo optimizar el uso del agua, sino también prevenir problemas como el desabastecimiento, el desperdicio o el sobregasto económico. La elección de tecnologías de bajo costo y libre acceso, como el ESP32 y las plataformas de desarrollo multiplataforma, hace que el sistema sea replicable y escalable en diferentes entornos. Además, la capacidad de analizar el historial de consumo permite al usuario tomar decisiones informadas sobre su uso del recurso.

Objetivos

Objetivo general:

Diseño e implementación de un sistema inteligente de monitoreo de nivel de agua

Objetivos específicos:

- Identificar los diferentes componentes y herramientas tecnológicas para el prototipo del sistema inteligente de monitoreo de nivel de agua.
- Diseñar un sistema que permita la medición remota del nivel de agua de un tanque utilizando un sensor ultrasónico y un ESP32 y además desarrollar una API que reciba y almacene los datos en una base de datos MongoDB
- Desarrollar una aplicación móvil que permita configurar los parámetros del dispositivo como dimensiones del tanque, red Wi-Fi y tarifa de agua y además

mostrar el nivel actual de agua, el porcentaje de llenado y el consumo histórico diario, semanal y mensual.

- Evaluar el buen funcionamiento y efectividad del prototipo y la aplicación mediante simulaciones.

Alcance del producto

El sistema permite medir y visualizar en tiempo real el nivel de agua en un tanque, calcular el volumen en litros y consultar el historial de consumo. Está orientado a entornos domésticos, agrícolas o institucionales donde se utilicen tanques de almacenamiento. El sistema admite la configuración de tanques de forma cilíndrica o cónica, permite registrar la tarifa por metro cúbico del servicio de agua, y presenta los datos a través de una aplicación móvil intuitiva. No contempla, en su versión actual, la automatización del llenado o vaciado del tanque, ni el control remoto de válvulas o bombas.

Diseño del producto

Oportunidad y Necesidad

Contexto del problema identificado

En diversos contextos, especialmente rurales y residenciales, el almacenamiento de agua en tanques es común. Sin embargo, muchas veces se desconoce el nivel real del agua y el consumo histórico de dichos tanques, lo cual puede llevar a desperdicio, sobrellenado, desabastimiento y altos costos. A pesar del avance tecnológico, en muchas regiones no se cuenta con soluciones de monitoreo accesibles que permitan gestionar eficientemente este recurso. Esta problemática representa una oportunidad para diseñar un sistema de bajo costo que permita visualizar en tiempo real el nivel de agua en un tanque, conocer el consumo y así tomar decisiones informadas.

Metodología utilizada para detectar necesidades

Para la identificación de necesidades, se utilizó la técnica de lluvia de ideas descrita en Ulrich y Eppinger (2021), donde cada integrante del equipo propuso una necesidad del entorno. Además, se realizaron entrevistas informales a usuarios potenciales como residentes de zonas rurales y urbanas que utilizan tanques de almacenamiento.

Resultados de entrevistas

- Los usuarios indicaron que frecuentemente no saben cuánta agua queda en sus tanques.
- A la hora de llenar los tanques los llenan sin tener una medida precisa ya que el agua llega en horarios específicos lo cual los hace llenar sin tener conocimientos del nivel.

- Existió un interés común en conocer el historial de consumo para en base a ello detectar consumos irregulares con lo cual se puedan detectar fugas o mejorar el uso eficiente del agua.
- Los usuarios manifestaron que querían conocer cuánto se gastaban en plata individualmente sus tanques.

Tabla de necesidades funcionales y no funcionales

Tipo de necesidad	Descripción
Funcional	Medir el nivel de agua en litros en tiempo real
Funcional	Calcular el volumen según la forma del tanque (cilíndrico, cónico)
Funcional	Mostrar los datos en una app móvil
Funcional	Almacenar los datos para historial de consumo diario, semanal y mensual
No funcional	Bajo costo de implementación
Funcional	Fácil instalación y mantenimiento
No funcional	Interfaz intuitiva para el usuario

Planeación del proyecto

Estructura del equipo y roles

- Cristian Ortega Montes: desarrollo de la API y la aplicación móvil.
- Javier Ortega Montes: ayuda en la creación de nuevas ideas del prototipo como la selección de los componentes y ensamblaje de los componentes.
- Brayan Bertel Álvarez: diseño de la carcasa y soporte en pruebas físicas.

Cronograma

Se utilizo un enfoque hibrido entre el modelo tradicional Gantt junto con elementos de SCRUM. Logrando así poder dividir las tareas en esprints de semanas.

Semana	Tarea
1	Lluvia de ideas y definición del problema
2	Investigación de tecnologías
3	Desarrollo inicial del prototipo físico
4	Integración de sensores y validación de medición
5	Desarrollo básico de la API junto con la app y pruebas
6	Diseño de la carcasa del prototipo
7	Pruebas finales e informe

Recursos disponibles y limitaciones

Recursos:

- Placa ESP32, sensor ultrasónico HC-SR04, protoboard, placa perforada, cables, carcasa impresa en 3D.
- Software: Node.js, MongoDB, Expo, React Native, Visual Studio Code y Arduino IDE.

Limitaciones:

- Presupuesto limitado
- Acceso restringido a impresoras 3D
- Tiempo acotado por responsabilidades y cargas académicas.

Riesgos identificados

- Fallas en la comunicación entre el microcontrolador y la API por conexión Wi-Fi inestable.
- Baja precisión en la medición de niveles por interferencias
- Posible daño de sensor ultrasónico por la humedad del interior del tanque.
- Complejidad en la instalación por parte de los usuarios sin experiencia técnica
- Fallas en el servidor por saturación
- La necesidad de romper la tapa del tanque para la instalación del prototipo podría causar desinterés en el producto.

Generación y selección de conceptos

Soluciones alternativas propuestas

1. Sistema con sensores infrarrojo y uso de pantalla LCD.
2. Sistema con sensor de presión en la base del tanque.
3. Sistema con sensor ultrasónico y conectividad Wi-Fi más aplicación móvil.

Matriz de decisión

Criterio	Peso	Opción 1	Opción 2	Opción 3
Costo	0.3	5	3	5
Precisión	0.25	2	5	4
Escalabilidad	0.2	2	3	5
Facilidad de uso	0.15	3	3	5
Instalación sencilla	0.1	3	4	4
Puntaje Total		3.15	3.6	4.65

Concepto seleccionado

Sistema con sensor ultrasónico y conectividad Wi-Fi más aplicación móvil: Elegido por su precisión, facilidad de instalación, bajo costo y escalabilidad. Permite un monitoreo remoto desde el celular mediante una API.

Bocetos o representaciones graficas del concepto elegido

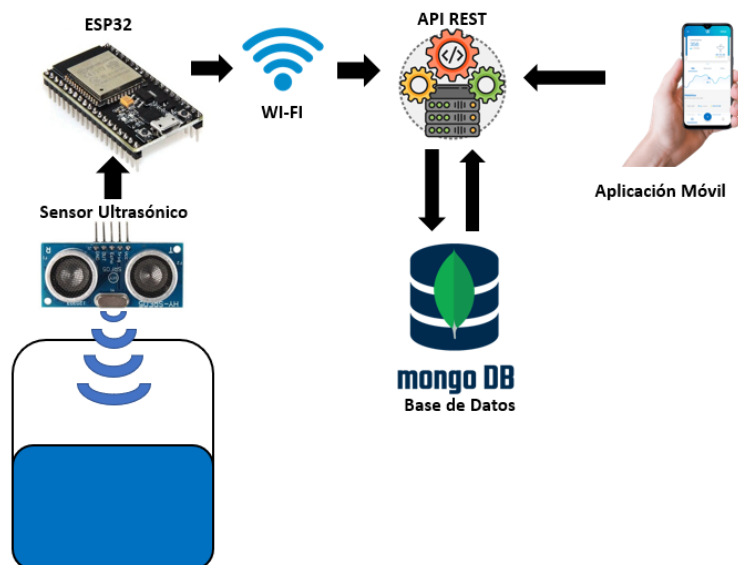


Figura 1. Representación del sistema.

Especificaciones del producto

Lista de requerimientos técnicos y de desempeño

- Separación mínima del sensor con respecto al nivel máximo del tanque: 3 cm
- Distancia de medición: 3 cm a 400 cm
- Transmisión de datos via Wi-Fi cada 90 segundos.
- Calculo automático del volumen en litros según la geometría del tanque.

Condiciones de operación esperadas

- Instalación en tanques a cielo abierto o bajo techo
- Buen funcionamiento en zonas con bastante humedad
- Temperatura operativa: 0 °C a 50 °C
- Alimentación directa a toma corrientes de 120 V AC

Especificaciones iniciales vs refinadas

Parámetro	Inicial	Refinado
Tipo de sensor	Sensor infrarrojo FC-51	Ultrasónico Hc-SR04
Visualización de datos	Pantalla LCD	App móvil con expo
Frecuencia de censado	Cada 60 segundos	Cada 90 segundos
Nivel de liquido	Distancia por metros	Nivel por litros
Configuración del dispositivo	Bluetooth	Wi-Fi

Tabla de criterios de validación

Criterio	Método de validación
Precisión de medición	Pruebas comparadas con regla milimétrica
Conectividad	Pruebas en red Wi-Fi domestica
Usabilidad de la app	Pruebas con usuarios
Registro de datos	Validación en MongoDB
Conexión con la API	Validación directa en el servidor

Desarrollo técnico (desde la ingeniería Electrónica)

Arquitectura del producto

Diagrama en bloques del sistema

El sistema de monitoreo de nivel de agua está compuesto por los siguientes módulos principales:

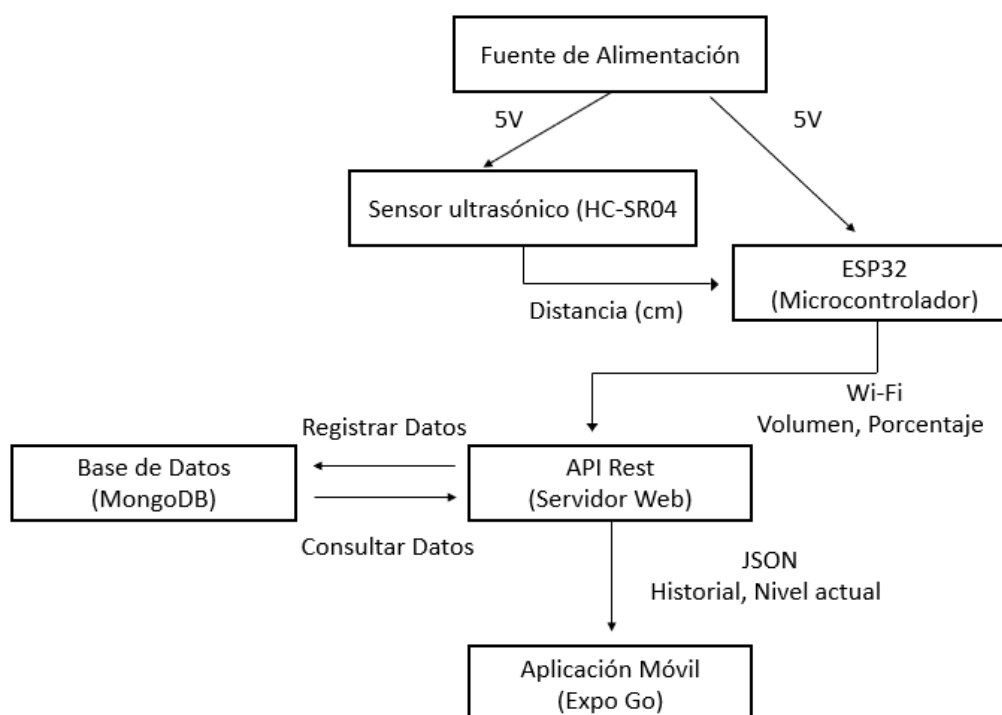


Figura 2. Diagrama de bloques del sistema.

Descripción de módulos o subsistemas

- **Modulo censado:** El sensor ultrasónico HC-SR04 mide la distancia entre el nivel del agua y el sensor, el cual está ubicado en la parte superior del tanque. Para obtener una

medición precisa, se considera la distancia mínima entre el nivel máximo del agua y el sensor, conocida como punto muerto de operación.

- **Unidad de procesamiento y comunicación:** El ESP32 procesa los datos del sensor, calcula el nivel de llenado según el volumen del tanque y el porcentaje, y envía esta información via Wi-Fi a la API.
- **Servidor:** La API recibe los datos y es el encargado de procesarlos, guardarlos y mostrarlos respectivamente según el tipo de petición que se le haga.
- **Subsistema de interfaz de usuario:** Incluye la app en React Native mediante Expo Go, donde los usuarios pueden registrar y configurar los dispositivos y además pueden visualizar el nivel de agua, historial de consumo y costos.

Flujo de datos / energía

- La energía fluye desde una fuente de alimentación externa de AC hacia el ESP32 y al sensor ultrasónico.
- Los datos fluyen desde el sensor ultrasónicos hacia el ESP32, y posteriormente desde este hacia la API mediante conexión Wi-Fi.

Consideraciones de diseño modular y escalabilidad

El sistema ha sido diseñado de forma modular, lo que permite reemplazar o mejorar componentes individuales sin afectar la arquitectura general. Por ejemplo, es posible cambiar el tipo de sensor o ampliar las funciones de la aplicación móvil sin modificar el núcleo del sistema. Además, el diseño admite la conexión de múltiples tanques o sensores a una misma red de monitoreo. Este sistema también es escalable, ya que puede integrarse con paneles solares para favorecer el ahorro energético y una mayor autonomía.

Diseño Electrónico

Esquemático del circuito

Se diseñó un circuito básico entre el sensor HC-SR04, el ESP32 y la fuente de alimentación. Las conexiones se realizaron directamente en una placa perforada como lo podemos ver en la Figura 3, por lo tanto, no se desarrolló un diseño de PCB.

Estas conexiones son las siguientes:

- **Fuente de alimentación:** V+ conectado al Vin del ESP32 y Vcc del sensor ultrasónico, GND conectado al GND del ESP32 y del sensor.
- **ESP32:** Pin D13 conectado al pin Tring del sensor ultrasónico y el pin D12 al pin Echo

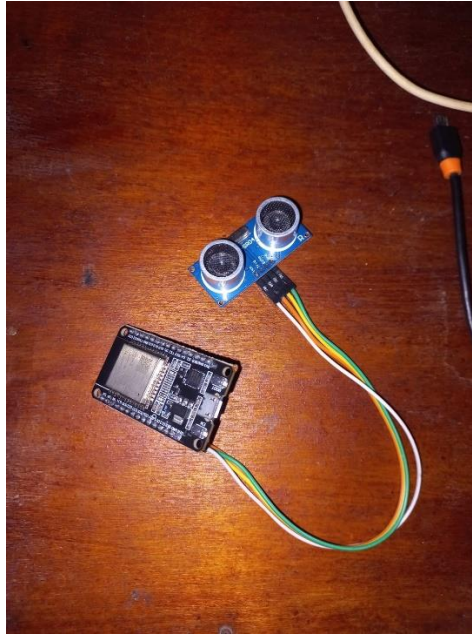


Figura 3. Conexiones del prototipo.

Selección y justificación de componentes

- **ESP32:** Se eligió este microcontrolador por su capacidad de procesamiento, bajo consumo de energía y conectividad Wi-Fi integrada, lo cual permite una comunicación eficiente con la API para enviar datos en tiempo real. Además, su compatibilidad con múltiples sensores lo hace ideal para proyectos IoT.
- **HC-SR04:** Sensor ultrasónico ampliamente utilizado debido a su buena precisión, bajo costo y facilidad de integración con microcontroladores como el ESP32. Es adecuado para medir la distancia entre el sensor y el nivel del agua dentro de un tanque.
- **Protoboard y cables Dupont:** Se utilizaron durante la fase de prototipado para realizar conexiones rápidas y temporales sin necesidad de soldadura, facilitando las pruebas y ajustes en el circuito.
- **Placa perforada (PCB universal):** Una vez validado el prototipo, el sistema fue montado sobre una placa perforada para asegurar las conexiones de forma permanente mediante soldadura. Esto mejora la durabilidad del montaje y la estabilidad del sistema en entornos reales.
- **Node.js (API Backend):** Se utilizó Node.js para el desarrollo de la API debido a su eficiencia en la gestión de múltiples conexiones simultáneas y su ecosistema rico en librerías. Esto permite recibir, procesar y almacenar los datos provenientes del ESP32 de forma rápida y escalable.
- **MongoDB (Base de datos):** Se eligió MongoDB por su estructura flexible basada en documentos, lo cual permite almacenar datos como registros de nivel de agua, fechas, horas y estados del sistema de forma eficiente. Además, su integración con Node.js es sencilla y potente.
- **Expo Go (Aplicación móvil):** Se utilizó Expo Go como entorno de desarrollo para React Native, lo cual permite construir una aplicación móvil multiplataforma de

forma rápida. La app permite visualizar en tiempo real el nivel de agua, recibir notificaciones, y gestionar múltiples sensores conectados a la red.

Consideraciones de compatibilidad y protección

En este proyecto no se incorporaron protecciones adicionales como diodos, fusibles o reguladores externos, debido a que el sistema es de baja complejidad y los componentes utilizados ya cuentan con ciertas protecciones integradas. Por ejemplo, el ESP32 incluye protección contra sobrecalentamiento y control de tensión, y el sensor ultrasónico HC-SR04 opera en niveles seguros de voltaje (5V), compatibles con el microcontrolador.

Además, el sistema fue diseñado para funcionar en un entorno controlado, lo que reduce el riesgo de sobrecargas o condiciones extremas. No obstante, en futuras implementaciones o aplicaciones en entornos más exigentes, se recomienda considerar protecciones adicionales para garantizar una mayor robustez y durabilidad del sistema.

Programación y control

Descripción de la lógica de control

La programación del sistema se diseñó con el objetivo de medir periódicamente el nivel de agua en un tanque utilizando el sensor ultrasónico HC-SR04, procesar estos datos con el microcontrolador ESP32 y enviarlos a una API desarrollada en Node.js. La aplicación móvil desarrollada con React Native (mediante Expo Go) permite visualizar los datos en tiempo real.

La lógica principal del sistema se basa en los siguientes pasos:

1. Activar el sensor ultrasónico y obtener el tiempo de ida y vuelta del pulso ultrasónico.
2. Calcular la distancia entre el sensor (ubicado en la parte superior del tanque) y la superficie del agua.
3. Determinar el nivel del agua restando la distancia medida a la altura total del tanque.
4. Enviar los datos mediante una solicitud HTTP POST al servidor backend.
5. Mostrar los datos en la aplicación móvil, junto con la fecha y hora de la medición.
6. Repetir el proceso de forma periódica con un retardo definido.

Algoritmo principal (pseudocódigo)

```
Inicio
  Configurar pines del sensor HC-SR04
  Recibir configuraciones
  Conectarse a la red Wi-Fi
  Mientras (true)
    Activar el pin de disparo (TRIG)
    Esperar respuesta en el pin de eco (ECHO)
    Medir duración del pulso
    Calcular distancia en cm
    Calcular nivel de agua = altura_total - distancia
    Calcular el porcentaje = (Nivel / Nivel_Total) * 100
    Enviar datos (nivel, porcentaje, id) a la API usando HTTP POST
    Esperar un intervalo
  FinMientras
Fin
```

Lenguaje de programación y entorno utilizado

- Firmware del ESP32: Se programó utilizando C++ en el entorno Arduino IDE, facilitando la comunicación con el sensor HC-SR04 y el envío de datos a través de HTTP.
- API Backend: Desarrollada en JavaScript usando Node.js con Express.js como framework. Se encarga de recibir, almacenar y gestionar los datos enviados por el ESP32.
- Base de datos: Se utilizó MongoDB para almacenar los datos del sistema de forma no relacional, permitiendo flexibilidad en el manejo de registros.
- Aplicación móvil: Se creó con React Native en el entorno Expo Go, permitiendo una interfaz amigable para el monitoreo en tiempo real desde dispositivos Android o iOS.

Fragmentos clave del código fuente

Código del ESP32 (Arduino):

```
#define TRIG_PIN 13
#define ECHO_PIN 12
#define BUTTON_PIN 14 // Pin del botón de reinicio
#define LED_PIN 15 // Indicador
String ssid, password, tipoTanque, deviceName, idDispositivo, idUsuario;
int alturaTanqueCM = 0; int ancho = 0; int anchoS = 0; int anchoB = 0; int separacionD = 0;
void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT); pinMode(BUTTON_PIN, INPUT_PULLUP); pinMode(LED_PIN, OUTPUT);
  preferences.begin("wifiConfig", false);
  ssid = preferences.getString("ssid", ""); password = preferences.getString("password", "");
  tipoTanque = preferences.getString("tipoTanque", ""); alturaTanqueCM = preferences.getInt("altura", 0);
  ancho = preferences.getInt("ancho", 0); anchoS = preferences.getInt("anchoS", 0);
  anchoB = preferences.getInt("anchoB", 0); separacionD = preferences.getInt("separacionD", 0);
  idDispositivo = preferences.getString("id", ""); idUsuario = preferences.getString("idUsuario", "");
  preferences.end();
  if (tipoTanque == "" || idDispositivo == "") {
    digitalWrite(LED_PIN, HIGH);
    iniciarModoConfiguracion();
  } else {
    conectarWifiYComenzar();
  }
}
void loop() {
  server.handleClient();
  if (modoConfiguracion) {
    static unsigned long lastBlink = 0;
    static bool ledState = false;
    if (millis() - lastBlink > 500) {
      ledState = !ledState;
      digitalWrite(LED_PIN, ledState ? HIGH : LOW);
    }
  }
}
```

```

    lastBlink = millis();
  }
}

if (digitalRead(BUTTON_PIN) == LOW) {
  if (!isButtonPressed) {
    buttonPressStartTime = millis();
    isButtonPressed = true;
  } else if (millis() - buttonPressStartTime >= 3000) {
    reiniciarConfiguracion();
  }
} else {
  isButtonPressed = false;
}

if (WiFi.status() == WL_CONNECTED && alturaTanqueCM > 0 && millis() - lastSampleTime > sampleInterval) {
  int distancia = medirDistancia(); int distanciaMaxima = alturaTanqueCM + separacionD;
  distancia = constrain(distancia, 0, distanciaMaxima);
  // Calcular altura útil de agua considerando el punto muerto
  float alturaAgua = distanciaMaxima - distancia; alturaAgua = constrain(alturaAgua, 0, alturaTanqueCM);
  float volumenLitros = 0.0; float volumenTotal = 0.0; float porcentaje = 0.0;
  if (tipoTanque == "Cilindrico") {
    float radio = ancho / 2.0;
    volumenLitros = PI * radio * radio * alturaAgua * 0.001;
    volumenTotal = PI * radio * radio * alturaTanqueCM * 0.001;
  } else if (tipoTanque == "Conico") {
    float R1 = anchoS / 2.0;
    float R2 = anchoB / 2.0;
    volumenLitros = (PI * alturaAgua * (R1 * R1 + R1 * R2 + R2 * R2)) / 3.0 * 0.001;
    volumenTotal = (PI * alturaTanqueCM * (R1 * R1 + R1 * R2 + R2 * R2)) / 3.0 * 0.001;
  }
  if (volumenTotal > 0) {
    porcentaje = (volumenLitros / volumenTotal) * 100.0;
  }

  // Enviar el volumen en litros como nivel
  enviarNivel(volumenLitros, porcentaje);
  lastSampleTime = millis();
}

}

int medirDistancia() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long duracion = pulseIn(ECHO_PIN, HIGH);
  return duracion * 0.034 / 2;
}

void enviarNivel(float volumenLitros, int porcentaje) {
  if (idDispositivo == "") {
    Serial.println("ID del dispositivo no configurado.");
    return;
  }
  HTTPClient http; http.begin(dataServer); http.addHeader("Content-Type", "application/json");
  DynamicJsonDocument doc(256);
  doc["id"] = idDispositivo; doc["nivel"] = volumenLitros; doc["porcentaje"] = porcentaje;
  String payload; serializeJson(doc, payload); int res = http.POST(payload);
  if (res > 0) {
    Serial.println("Datos enviados correctamente.");
    Serial.println(payload);
  } else {
    Serial.printf("Error al enviar datos. Código HTTP: %d\n", res);
  }
  http.end();
}

```

Figura 4. Código principal de ESP32

Fragmento del backend en Node.js (Express):

```

//Recibir y Registrar Nivel
NivelCtrl.Nivel = async (req, res) => {
  try {
    const { nivel, id, porcentaje } = req.body;

    if (nivel === undefined || nivel === null || !id) {
      return res.status(400).json({ mensaje: "Datos incompletos" });
    }

    // Crear documento con fecha actual (usando moment)
    const nuevoNivel = new Nivel({
      idDispositivo: id,
      Nivel: nivel,
      porcentaje: porcentaje,
      FechaSensado: moment().startOf("day").toDate(), // convierte a objeto Date
    });

    await nuevoNivel.save();

    res.status(201).json({ mensaje: "Nivel registrado correctamente" });
  } catch (error) {
    console.error("Error al registrar nivel:", error);
    res.status(500).json({ mensaje: "Error en el servidor" });
  }
};

//Mostrar Nivel
NivelCtrl.DetallesDeNivel = async (req, res) => {
  try {
    const { dispositivoId } = req.params;

    const dispositivos = await SmartTankConf.find({
      idDispositivo: dispositivoId,
    }).lean();

    const ultimoNivel = await Nivel.findOne({ idDispositivo: dispositivoId })
      .sort({ createdAt: -1 })
      .exec();

    if (!ultimoNivel) {
      return res
        .status(404)
        .json({ mensaje: "No se encontraron datos para este dispositivo" });
    }

    // Calcular fecha hace 7 días
    const hoy = new Date();
    const hace7Dias = new Date();
    hace7Dias.setDate(hoy.getDate() - 7);

    // Obtener últimos niveles de los últimos 7 días (uno por día)
    const nivelesUltimos7Dias = await Nivel.aggregate([
      {
        $match: {
          idDispositivo: dispositivoId,
          FechaSensado: { $gte: hace7Dias, $lte: hoy }
        }
      },
    ]),

    {
      $project: {
        Nivel: 1,
        FechaSensado: 1,
        dia: {
          $dateToString: { format: "XY-Xm-Xd", date: "$FechaSensado" }
        }
      },
    },
    {
      $sort: { FechaSensado: -1 }
    },
    {
      $group: {
        _id: "$dia",
        Nivel: { $first: "$Nivel" },
        FechaSensado: { $first: "$FechaSensado" }
      },
    },
    {
      $sort: { FechaSensado: 1 } // Para ordenarlos cronológicamente
    }
  ]);

  res.json({ nivel: ultimoNivel.Nivel, porcentaje: ultimoNivel.porcentaje, dispositivos, nivelesUltimos7Dias });
} catch (error) {
  console.error("Error al obtener nivel:", error);
  res.status(500).json({ mensaje: "Error en el servidor" });
}
};

```

Figura 5. Código principal de la API.

Fragmento de la app móvil en React Native (Expo Go):

```
//consultar nivel
useEffect(() => {
  const fetchNivel = async () => {
    try {
      const res = await fetch(
        `http://192.168.1.8:4000/Detalles/nivel/${dispositivoId}`
      );
      const data = await res.json();
      setNivel(data.nivel);
      setporcentajeNivel(data.porcentaje);
      setalturaTanque(data.dispositivos[0].HeightTank);
      setNivelesHistoricos(data.nivelesUltimos7Dias);
      setError(null);
    } catch (error) {
      console.error("Error al obtener nivel:", error);
      setError("Error al obtener nivel.");
    }
  };

  fetchNivel();
  const interval = setInterval(fetchNivel, 1000);
  return () => clearInterval(interval);
}, []);
```

Figura 6. Código principal de la App.

Validación, Prototipado y Resultados

Prototipado

Para validar el funcionamiento del sistema de monitoreo de nivel de agua, se realizaron prototipos físicos funcionales, utilizando componentes electrónicos comerciales como el sensor ultrasónico HC-SR04 y el microcontrolador ESP32. No se diseñaron PCBs a medida, ya que se optó por utilizar placas de desarrollo listas para prototipado con conexión por pines tipo jumper.

Tipo de prototipos realizados

- **Prototipo con Arduino y pantalla LCD:** el cual permitió medir la distancia entre el sensor y el nivel de agua permitiendo así conocer la altura del agua en cm.

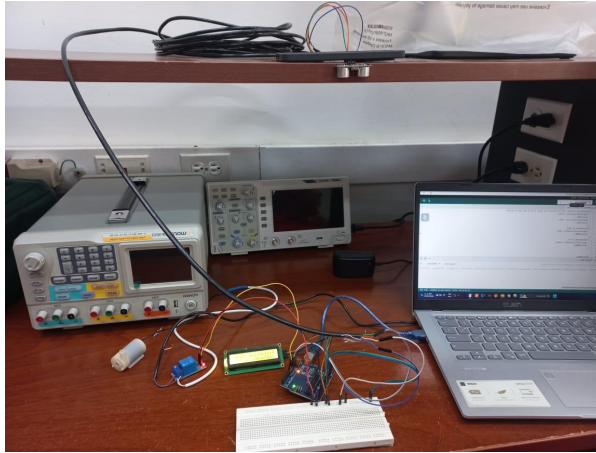


Figura 7. Primer prototipo con Arduino

- **Prototipo con ESP32:** Permitió verificar la lectura de distancias, el procesamiento de datos en el ESP32, la conexión Wi-Fi y la comunicación con la API.

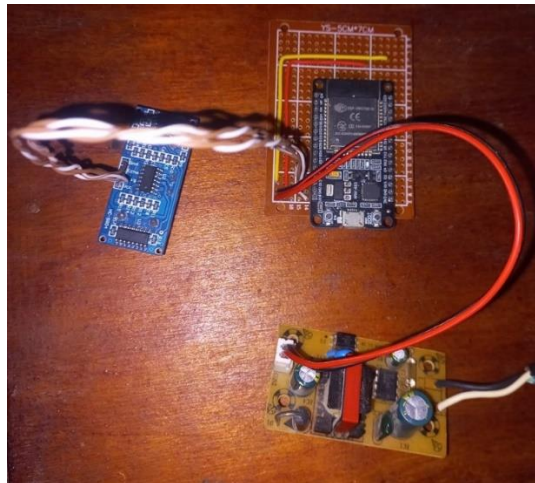


Figura 8. Segundo prototipo con ESP32.

- **Prototipo de software:** Incluyó la aplicación móvil desarrollada en **Expo Go**, con una interfaz extremadamente sencilla conectada en tiempo real con la API que almacena los datos en MongoDB. Para una visualización rápida del nivel en metros en tiempo real

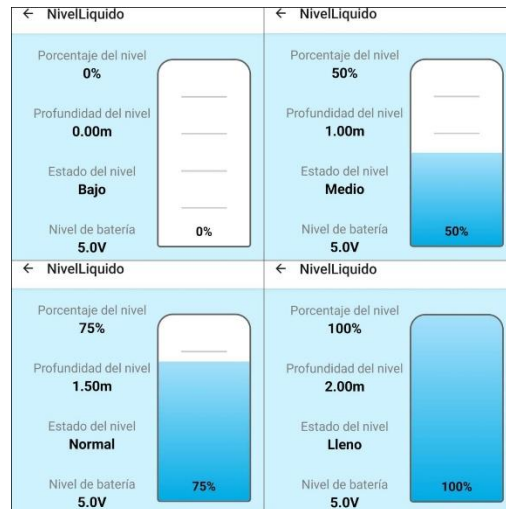


Figura 9. Primera Visualización de datos.

Iteraciones de diseño

Durante el desarrollo, se realizaron múltiples iteraciones para mejorar el rendimiento y la estabilidad del sistema:

- Se ajustó el rango de lectura del sensor ultrasónico para adaptarlo a tanques de diferentes alturas.
- Se modificó el cálculo de nivel, se pasó a mostrarlo en litros en vez de metros.
- Se optimizó la lógica de cálculo de volumen para permitir formas cilíndricas y cónicas.
- Se modificaron elementos de la interfaz de la app móvil para mejorar la visualización de datos históricos.

Montaje, pruebas preliminares y ajustes

El sistema fue ensamblado dentro de un contenedor plástico, con una fuente de alimentación mediante una batería de 5V DC, lo que garantiza portabilidad y aislamiento de fuentes de alimentación externas. Posteriormente, el módulo fue instalado en la parte superior de un tanque de prueba, específicamente un tanque doméstico de cocina con forma cónica y de gran volumen.

Durante la fase de pruebas preliminares, se realizaron mediciones continuas del nivel de agua y se verificó el correcto envío de datos hacia la API mediante la red Wi-Fi. Los resultados fueron satisfactorios en cuanto a conectividad y respuesta del servidor, confirmando que el flujo de datos entre el ESP32, la API Node.js y la aplicación móvil fue exitoso y estable.

No obstante, se identificaron interferencias en la lectura del sensor ultrasónico cuando el tanque contenía elementos flotantes (como utensilios o residuos) o cuando su superficie estaba parcialmente cubierta. Estas condiciones generaban lecturas inconsistentes o erróneas. Para mitigar estos efectos, se implementaron dos soluciones:

- **Promedio de lecturas múltiples:** Se toman varias mediciones por ciclo, y se calcula el promedio para reducir el error causado por obstáculos momentáneos o superficies irregulares.
- **Manejo de distancias mínimas (punto muerto de operación):** Se estableció un umbral fijo entre el sensor y el nivel máximo del agua para ignorar valores fuera del rango real de operación, evitando así falsos positivos en niveles altos.

Como resultado, el sistema mostró un comportamiento estable y confiable, tanto en la lectura como en la transmisión de datos, siendo capaz de monitorear eficazmente el nivel de agua en condiciones reales.



Figura 10. Sistema montado.

Pruebas del Producto

Procedimientos de validación

Para garantizar el correcto funcionamiento del sistema, se definieron y ejecutaron diversos procedimientos de validación, enfocados tanto en la precisión de las mediciones como en la integridad de la comunicación entre los distintos componentes. Las pruebas realizadas fueron las siguientes:

- **Comparación con mediciones manuales:** Se utilizaron cintas métricas para medir manualmente la distancia entre el sensor ultrasónico y el nivel del agua. Estas mediciones se compararon con las lecturas registradas por el sensor HC-SR04 para verificar su precisión. Los resultados mostraron una desviación de 0.5 cm de diferencia bastante aceptable dentro del margen de error esperado para este tipo de sensores.
- **Cálculo del volumen de agua:** Se calculó el volumen del agua en el tanque de forma manual (teniendo en cuenta su forma cónica) y se comparó con los valores estimados por el sistema, basados en el nivel medido. Esto permitió validar la consistencia del procesamiento de datos realizado por el microcontrolador y la aplicación.

- **Pruebas de conectividad Wi-Fi:** Se evaluó el comportamiento del sistema bajo diferentes condiciones de señal Wi-Fi, incluyendo cambios de ubicación del módulo. El ESP32 mantuvo una conexión estable en la mayoría de los entornos, mostrando una buena capacidad de transmisión de datos incluso con señales medias o débiles.
- **Verificación de la API y la app móvil:** Se realizaron pruebas de envío y recepción de datos hacia y desde la API desarrollada en Node.js. Además, se comprobó la correcta visualización de la información en la interfaz de la aplicación móvil (Expo Go), confirmando que los datos llegaban en tiempo real y sin errores de interpretación.

Estos procedimientos permitieron validar que el sistema cumple con los objetivos de medición, transmisión y visualización del nivel de agua en el tanque, con un comportamiento robusto y confiable en pruebas de campo.

Resultados Técnicos y Comparación con Especificaciones

Tras la implementación y validación del sistema, se recopilaron los siguientes resultados técnicos, los cuales fueron contrastados con las especificaciones teóricas esperadas para evaluar el rendimiento general del sistema.

1. Precisión en la medición de distancia

Las lecturas del sensor ultrasónico HC-SR04 se compararon con mediciones manuales realizadas con cinta métrica. La diferencia promedio entre ambas fue de aproximadamente **0.5 cm**, lo cual se considera un **error mínimo** y aceptable dentro del margen de tolerancia del sensor (± 3 mm típicos en condiciones ideales).

Método de medición	Valor (cm)	Diferencia
Medición manual	55.0	—
Lectura del sensor	54.50	-0.5

2. Estimación del volumen de agua

Utilizando la geometría del tanque (forma cónica) y el nivel de agua registrado, se estimó el volumen total de agua presente en el depósito. El sistema calculó un valor de 196.5 litros, mientras que el cálculo manual indicó 197 litros, obteniéndose así una diferencia de solo 0.5 litros, lo cual representa un error relativo del 0.25%, considerado muy bajo para aplicaciones domésticas.

Método de cálculo	Volumen estimado (litros)	Diferencia
Cálculo manual	197.0	—
Estimación del sistema	196.5	-0.5

3. Desempeño general

- **Transmisión de datos:** Exitosa, sin pérdidas detectadas durante las pruebas.
- **Visualización en app móvil:** Fluida, con actualización en tiempo real.
- **Precisión global del sistema:** Alta, dentro del margen de tolerancia técnica de los sensores y cálculos implementados.

Estos resultados confirman que el sistema cumple con los objetivos técnicos de precisión y confiabilidad establecidos al inicio del proyecto, siendo adecuado para monitoreo de niveles de agua en tanques domésticos o rurales.

Gráficas del desempeño

Se generaron graficas de desempeño entre la distancia real con la medida así como también el volumen en litros calculado manualmente como el medido

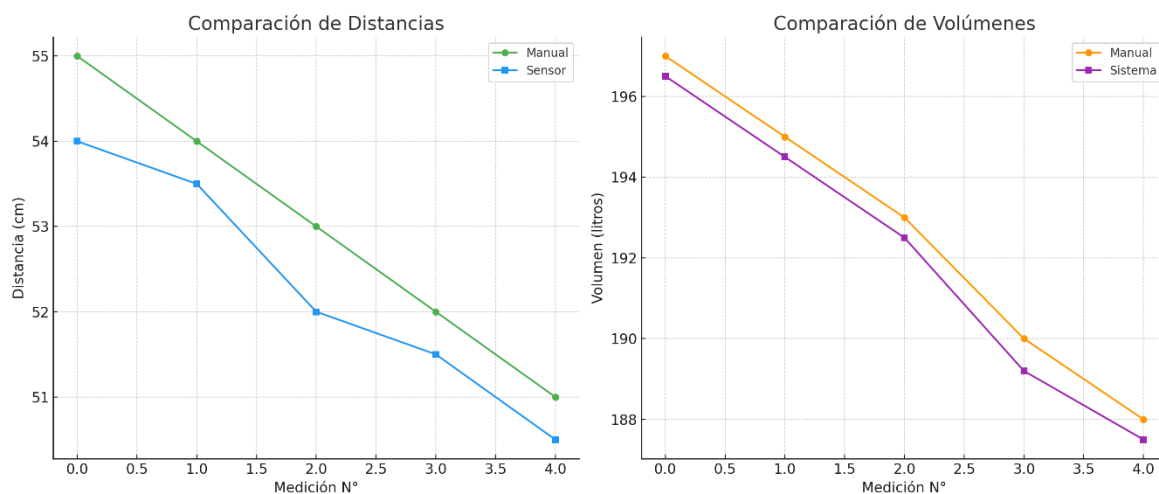


Figura 11. Valores reales Vs medidos.

Problemas Encontrados y Correcciones Realizadas

Durante el desarrollo e implementación del sistema, se identificaron algunas situaciones que afectaban el desempeño. A continuación, se describen los principales problemas y las soluciones adoptadas:

- **Interferencias en las lecturas del sensor:** Se observaron fluctuaciones en los datos cuando el tanque contenía objetos flotantes o estaba parcialmente cubierto.
Solución: Se implementó un algoritmo de promedio de múltiples lecturas para estabilizar los valores obtenidos y reducir el error.
- **Zonas muertas de medición:** El sensor presentaba lecturas erráticas al acercarse al nivel máximo del tanque, debido al punto ciego del HC-SR04.
Solución: Se definió un "punto muerto" o margen de operación seguro que evita tomar valores cercanos al límite físico del sensor.
- **Problemas iniciales de conectividad Wi-Fi:** En ciertas ubicaciones, el módulo ESP32 no lograba conectarse consistentemente.
Solución: Se configuraron rutinas de reconexión automática y se probó en entornos con mejor cobertura de red.

Estas correcciones permitieron mejorar la estabilidad y confiabilidad general del sistema, logrando un desempeño satisfactorio en pruebas reales.

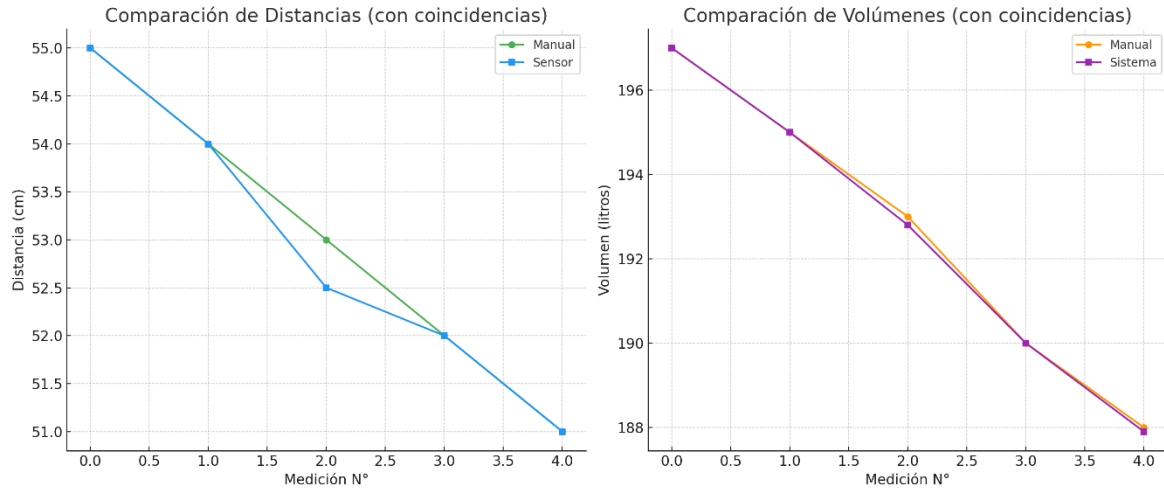


Figura 12. Mejora del desempeño.

Reflexión Final e Impacto

Dificultades encontradas y lecciones aprendidas

Dificultades técnicas

Durante el desarrollo del sistema de monitoreo de nivel de agua, se presentaron varios retos técnicos. Uno de los principales fue la lectura precisa del sensor ultrasónico HC-SR04, el cual en ciertas condiciones, como la presencia de humedad o superficies irregulares del agua, arrojaba valores inestables.

Otro desafío fue la conectividad Wi-Fi. La inestabilidad de la red en entornos reales afectó el envío constante de datos. Para mitigarlo, se incorporó una rutina de reconexión automática en caso de pérdida de señal. También se optimizó el tiempo de respuesta de la API desarrollada en Node.js, garantizando una comunicación eficiente con la base de datos MongoDB.

El manejo de formatos de datos JSON y la conversión de unidades físicas como distancia, volumen y porcentaje de llenado implicaron una curva de aprendizaje, que se superó con la consulta de documentación técnica y foros especializados.

Dificultades de diseño y trabajo en equipo

En la fase inicial del proyecto, uno de los principales retos fue la definición clara de roles y la comunicación efectiva entre los integrantes. Aunque el equipo estuvo conformado por personas con conocimientos diversos, la coordinación entre quienes desarrollaban el software (Cristian Ortega Montes) y quienes se encargaban del prototipo físico (Javier Ortega Montes y Brayan Bertel Álvarez) requería de una planificación detallada y sincronización constante. Además, el diseño del producto enfrentó limitaciones debido al uso de componentes electrónicos comerciales, lo que restringió algunas decisiones de integración y compactación del sistema. Sin embargo, esta limitación también permitió enfocarse en la funcionalidad y rapidez de implementación. Estas experiencias resaltaron la importancia del trabajo colaborativo, la gestión del tiempo y la documentación técnica común.

Impacto potencial del producto

Aplicaciones reales del producto

El sistema desarrollado tiene un gran potencial de aplicación en hogares, fincas, colegios, edificios residenciales y comunidades rurales, donde el monitoreo eficiente del nivel de agua puede ayudar a mejorar la gestión de los recursos hídricos. Además, permite llevar un control del consumo diario, lo que puede fomentar el uso responsable del agua, detectar fugas o excesos y reducir costos asociados. En zonas donde el suministro de agua no es constante, este sistema puede alertar a los usuarios sobre niveles críticos, evitando desabastecimientos y facilitando una mejor planificación del consumo.

Posibilidades de comercialización o mejora

El producto es altamente escalable y mejorable. Algunas de las oportunidades para su comercialización o desarrollo futuro incluyen:

- Integración con servicios de notificación como WhatsApp o Telegram.
- Interfaz web para visualizar datos desde un navegador.
- Versión con alimentación por panel solar para uso en zonas rurales.
- Carcasa profesional resistente al agua y con anclajes estándar.
- Inclusión de sensores adicionales como flujo de agua o presión.

Además, su estructura modular permite la fácil adaptación a distintos tipos y tamaños de tanques, lo que amplía su mercado potencial.

Reflexión del equipo: lecciones aprendidas y habilidades desarrolladas

A lo largo del proyecto, el equipo desarrolló habilidades técnicas en áreas como:

- Programación embebida en ESP32 (C++ / Arduino).
- Desarrollo de APIs RESTful en Node.js.
- Manejo de bases de datos NoSQL (MongoDB).
- Uso de herramientas de prototipado rápido (Expo Go, sensores ultrasónicos).
- Análisis de datos y visualización en tiempo real.

Desde una perspectiva personal y profesional, los integrantes del equipo también fortalecieron su capacidad de trabajo colaborativo, gestión de tareas, solución de problemas, y documentación técnica, elementos fundamentales para cualquier proyecto de ingeniería. El proyecto demostró que, con recursos limitados pero bien gestionados, se pueden lograr soluciones funcionales con impacto real en el entorno social y ambiental.

Referencias

Espressif Systems. (2022). *ESP32 technical reference manual* (v4.4).
<https://www.espressif.com/en/support/documents/technical-documents>

Expo. (2024). *Expo documentation*. <https://docs.expo.dev/>
FAO. (2023). *Gestión del agua: situación actual y desafíos futuros*. Organización de las Naciones Unidas para la Alimentación y la Agricultura. <https://www.fao.org>
Meta. (2024). *React Native documentation*. <https://reactnative.dev>
MongoDB Inc. (2024). *MongoDB documentation*. <https://www.mongodb.com/docs/>

Anexos

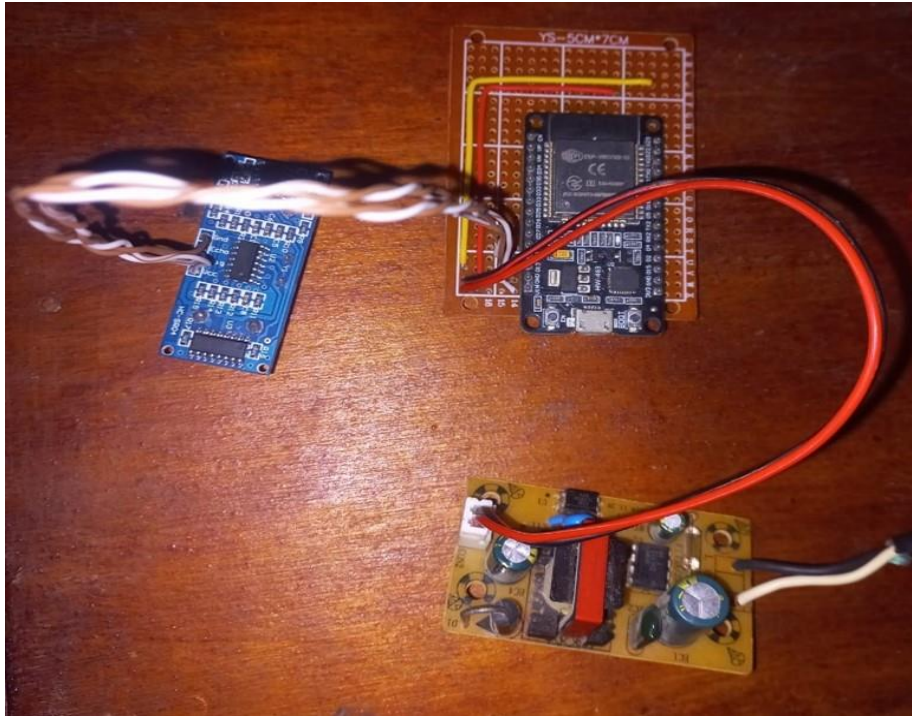


Figura 13. Todo conectado.

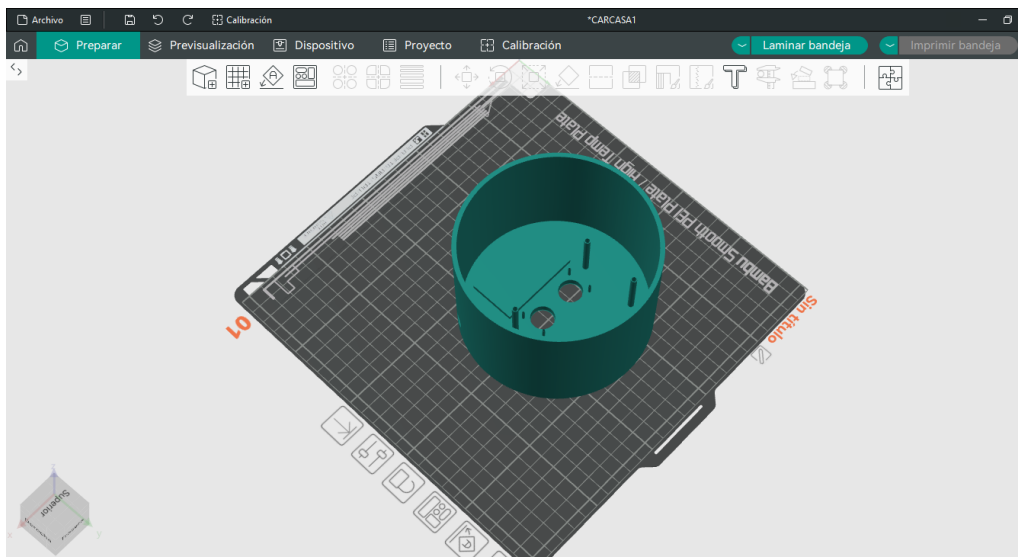


Figura 14. Diseño 3D de la carcasa.

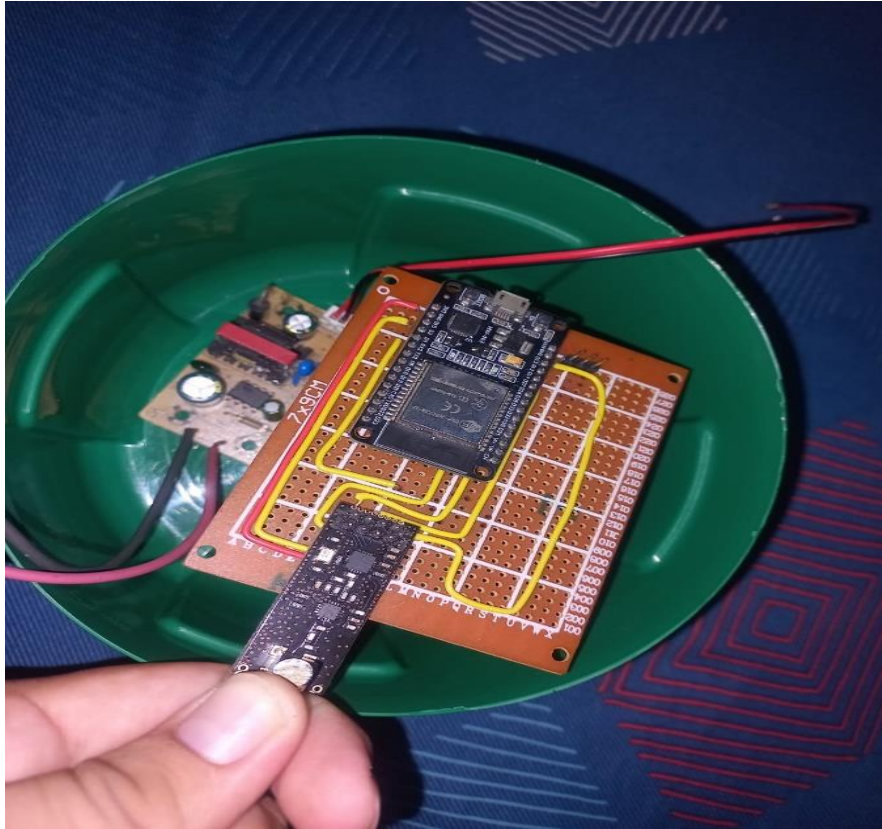


Figura 15. Primer montaje.

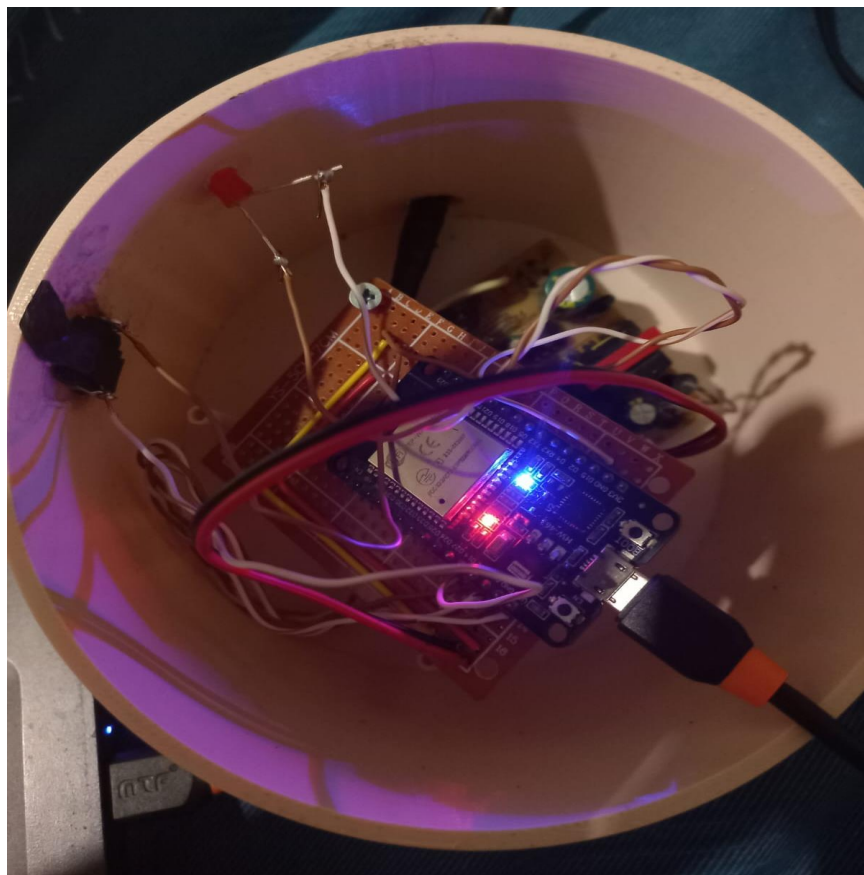


Figura 16. Ultimo prototipo.

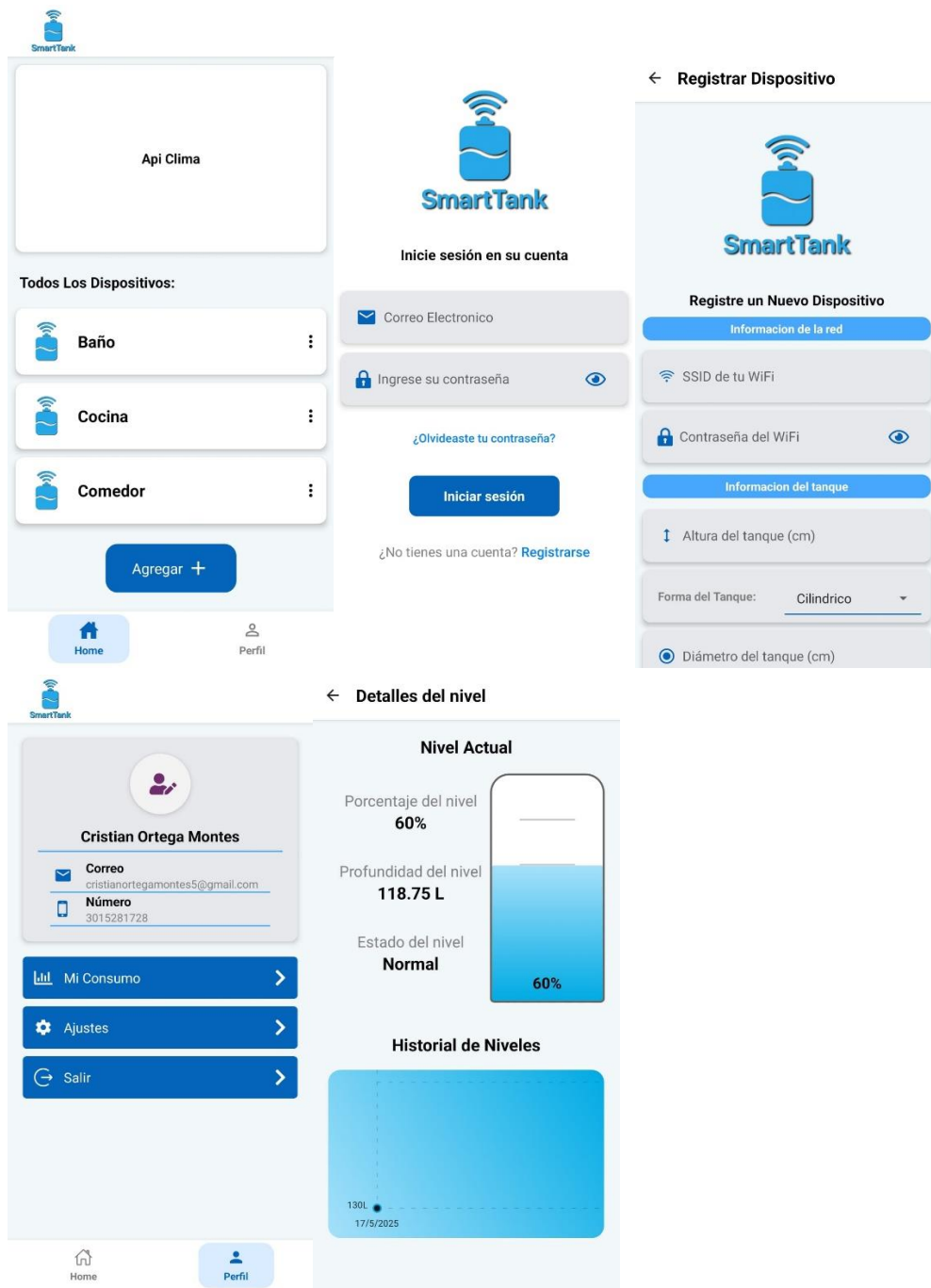


Figura 17. Vistas de la aplicación.

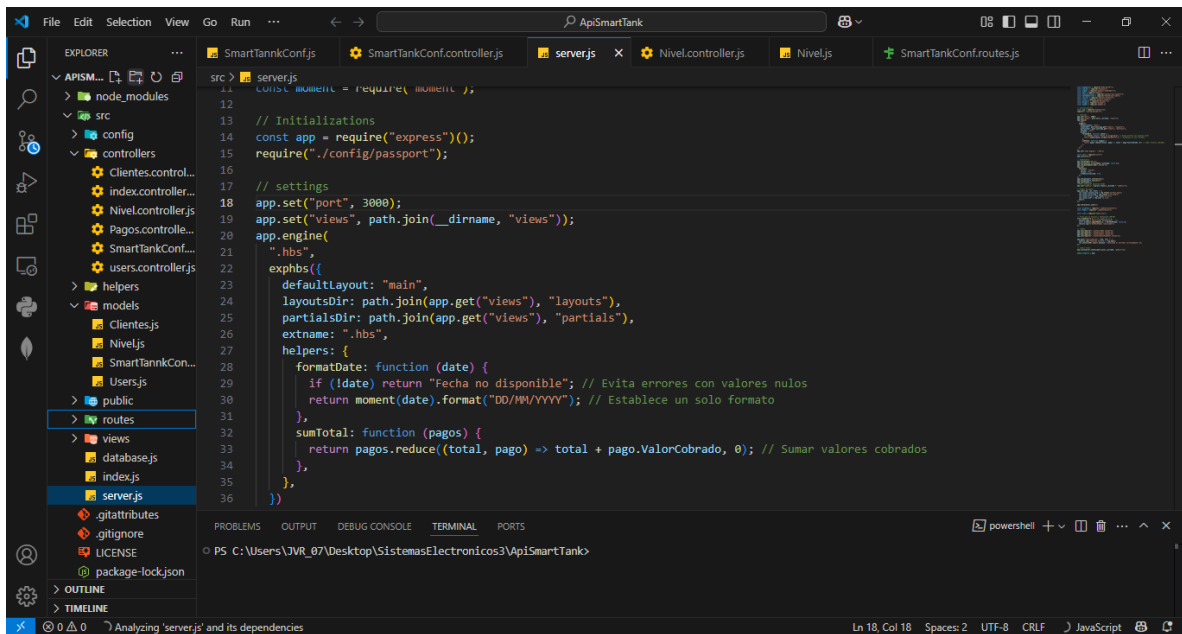


Figura 18. Estructura basica de la API.

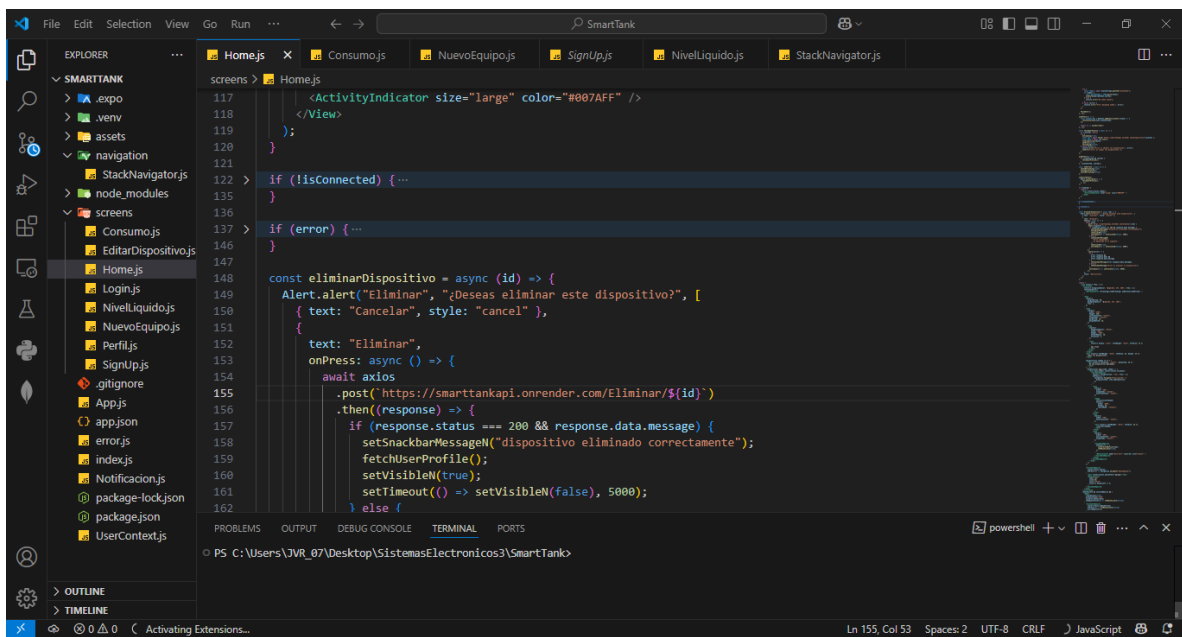


Figura 19. Estructura basica de la aplicación.

```

1  #include <WiFi.h>
2  #include <WebServer.h>
3  #include <HTTPClient.h>
4  #include <Preferences.h>
5  #include <ArduinoJson.h>
6
7  #define TRIG_PIN 13
8  #define ECHO_PIN 12
9  #define BUTTON_PIN 14 // Pin del botón de reinicio
10 #define LED_PIN 15 // Indicador
11
12 WebServer server(80);
13 Preferences preferences;
14
15 String ssid, password, tipoTanque, deviceName, idDispositivo, idUsuario;
16 int alturaTanqueCM = 0;
17 int ancho = 0;
18 int anchoS = 0;
19 int anchoB = 0;
20 int separacionD = 0;
21
22 const char* dataServer = "https://smarrtankapi.onrender.com/nivel";
23 const char* dataServerRegistrar = "https://smarrtankapi.onrender.com/registro";
24 unsigned long sampleInterval = 10000;
25 unsigned long lastSampleTime = 0;
26 unsigned long buttonPressStartTime = 0;
27 bool isButtonPressed = false;
28 bool modoConfiguracion = false;
29
30 > void setup() { ...
56 }
57
58 > void loop() { ...
120 }
121
122 > void iniciarModoConfiguracion() { ...
262 }
263
264 > void conectarWifiYComenzar() { ...
289 }
290
291 > void reiniciarConfiguracion() { ...
298 }
299
300 > int medirDistancia() { ...
308 }
309
310 > void enviarNivel(float volumenLitros, int porcentaje) { ...
337 }

```

Figura 20. Código fuente del dispositivo.